SASKEN

Systematic Approach and Guidelines for Cyber Security Testing

Whitepaper

Abstract:

This article explains about Cyber Security Testing approach which can be used in testing projects. This paper looks at possible best practices for coming up with cyber security testing. It explains how the increasing system complexity poses significant threats and challenges as well as how end-to-end security testing of the system with high level application software interacting among multiple modules can be achieved. This white paper talks about techniques such as penetration testing and fuzzing for ensuring effective end-to-end security testing during overall system development process.

Author:
Allappagoud Biradar, Architect-Product Testing,
Product Engineering Services, Sasken

Table of Content

Safety and Security

People and environment need to be protected against risks or issues generated by machines. This is safety, while security means protecting the machines against risk or issue generated by people. Earlier in automation, hacking was not easy because every system was insulated and capsuled. With tremendous innovation in Cloud, IoT, etc. more attacks are occurring because of wireless connectivity. We may not know that on the same node there is data for control purposes which people with a criminal bent of mind could try and use for some other unintended purpose. So there is a need to protect the system in order to minimize such attacks.

Motivation

There are different standards for safety and security. For example, IEC 61508 is a functional safety standard applicable to all kinds of industries. As the safety definition by v-model, how in the development of the system, in which phases, what measures, methods, and what topic of security can be addressed. We can address security in the safety development process because both should be addressed at the same time in order to save cost and effort. Once the development has progressed further, then it will be hard to implement security aspects.

As per our development environment, project activities involve multiple cycles, which leads to lot of time and cost. So a possible way to avoid this cost is to address the security testing permanently. In a development cycle, most of us in the v-model start doing security testing very late in the cycle. Therefore, development of safety and security aspects needs to be addressed right at the beginning of the development engineering process.

Development Phase: Requirements

At the beginning of analysis, we can look at the requirements. We have to do this from the safety aspects as per 26262 or as per 61508.Here are the requirement processes:

a. Security Requirement Analysis:

Range of methods:
1. General Accounting Office's (GAOs) models
2. Co-Benefits Risk Assessment (COBRA)

If we are in the requirement analysis phase, we can actually look at methods in order to show where there would be a possibility to get security requirements. Entire set of methods, the ones that we are using mostly or preferred in our practical applications depending on what circumstances are.

b. SQUARE (System Quality Requirements Engineering)

SQUARE is a process for definition of target levels of trust and certain processes and procedures in order to win requirements. Once we get the requirements, then we can address security. We can deal with this requirement just the same way as safety requirement or functional requirement. If we develop it accordingly, large part of security has been taken to consideration right from the beginning. Right at the very start of the requirement phase, the security activity needs to be started.

Development Phase: Design

a. Activity: Threat Modeling

Another possible way of doing it is little bit later in the development process, once we have requirements, and then try to develop the architecture that cover those requirements, and then look at threat modeling.

Threat modelling means that security architecture must think about what kind of actions could compromise the system if they were attached. So we generate threat models.

We can work step by step in the threat modelling process. We do the decomposition of the system and classify modules. This is exactly the same process as development model. Normally the way we look at functional decomposition process, the same approach needs to be applied for security aspect. We develop models, by way of these models, we try and find out if the architecture is good enough already. One of the important points is to come up with good architecture. Everything that comes downstream will suffer if the architecture has not incorporated correct actions.

- Application decomposition
- Definition and classification of Modules
- Analysis of potential weak points
- Analysis of potential threats
- Establish counter strategies

b. Threat Modeling Advantages

- Early stage of development
- Analysis and reflection of system like attackers will do

Particularly when it comes to threat modelling, we are in the early stage of development and have not yet spent too much time and resources in development. When it comes to cost, this can be developed with least cost. Now when we look at safety and security, this phase will extend and become longer. The drawback is some people rely on this, assuming we have done everything in the early phases to develop good threat models, but unfortunately this does not mean we have developed good software. If we develop threat models for the software, then we must analyze these outcomes and results, and then take the corresponding action.

Development Phase: Implementation

a. Activity: Static Code Analysis

Once we have developed threat models in the design phase, the next proven approach is static code analysis in the implementation phase. This may be done as manual reviews in many cases but there are standard tools also available. With tools getting better and better, these activities can be carried out in an automated way. If we forget something during the development process, such as selecting the wrong buffer size, they will be corrected automatically. This is something, somebody who is trained, read the code, and understands it correctly can do. A code review criterion for static code analysis is mentioned below:

- Buffer overruns and overflows
- OS Injection
- SQL Injection
- Data Validation
- Cross-Site Scripting
- Race Conditions

b. Advantages of Static Code Analysis

We have to use experts even when using automated static code analysis tools because these tools will not detect issues in the software. We have highly creative developers who keep open backdoors, assuming that these security issues can be taken up at later point of the development cycle. These backdoors are not just available to developers but to attackers as well. If we put these backdoors (for example master password) into a user manual and publish them, there are chances that we might have forgotten to close these backdoors thus leaving them wide open to possible attacks. Therefore, static analysis as a tool is a fantastic thing because it looks at every line of code. Here are few advantages of using static code analysis tools:

- Completeness
- Accuracy
- Speed

c. Disadvantages of static code analysis

The drawback however is, we need highly trained experts as reviewers. We need high skill sets to be able to understand the language of the line of code and specification. One needs to be an expert in the entire domain. Another drawback is, when it comes to these reviews, one only tends to review the new code. For whatever reasons, we tend to use libraries that we already have and nobody looks at older libraries. More often than not, there are problems sitting with older code than newer code. So, the drawback of static code analysis is, we only tend to look at the new code, without running the system. So we have only few class of errors which can be detected. If we were to go through regression tests, at some stage we need to develop loop during the development phase, so high effort is needed to change the code base.

Development Phase: Test

a. Activity: Penetration Testing

Moving over to dynamic security testing, in dynamic case we try that the system to be penetrated with whatever kind of mechanisms to produce bugs or find backdoors open to exploit something which designer or developer have not thought of.

In penetration testing, there is quite a lot happening. Of course, the drawback generally is, we have to do this on behalf of someone. Let's say the telecom system is unsecure, we will try and penetrate it. Though we need telecom commissioning to do this, we have to agree on what we are allowed to do and when we have to do; if we do something bad, how bad these tests are going to be. When it comes to legality of something, it is good to look at BSI (Germany government organization) recommended test suites such as OpenVAS (Open Vulnerability Assessment System) & NVTs (Network Vulnerability Tests). They have produced their own suits for penetration testing.

b. Penetration Testing Advantages

In penetration testing, if we will have a test suite available, then we can get it to run easily and quickly. We also don't need many highly skilled experts to get good results.

- Fast and cheap
- Lower demands on skill set
- Tests the code that is actually being exposed

c. Penetration Testing Disadvantages

One major disadvantage of penetration testing is that, this type of testing is carried out very late in the overall software development life cycle. The other disadvantage is that penetration testing is carried out only on black box system (front impact) and there will be no other sophisticated approaches to attack the system.

- Too late in software development life cycle
- Front impact testing only

Development Phase: Test

a. Activity: Fuzzing

Fuzzing tries to find security problems that were not known and nobody thought of. Fuzzing is the technology of choice when someone wants to find unknown security loopholes. There are two categories in fuzzing. One is Recursive fuzzing and other is Replacive fuzzing.

b. Fuzz Vector Category

1. Recursive fuzzing: Iterating through all the possible combinations of a 'set alphabet Recursive fuzzing is rather straight forward. We have a place where we have a data interface somewhere and all sort of combinations of input data are sent to that interface. So if we know the alphabet that is possible there, we will just try all possible combinations. Total possibility, after this the system was unable to be compromised. It takes a lot of time unless you have the possibility in certain environment with the high performance or may be corresponding number of your test systems to do things in parallel. This is being done because this is rather straight forward but again there are some limits to it as well.

2. Replacive fuzzing: Replacing with a 'set value' Fuzz vector: When it comes to using passwords, to which the normal user would not be thinking of as using there is a password. This would be put into a fuzz vector. We have different attacks that we subject the system to, some of these attacks are known to you.

i. Cross Site Scripting (XSS)

Normally the developer or designer is expected to say, I have entered text that represent name or password. Rather than this, we put script into this input line and we will check what system will do with it. With cross site scripting, we can go and try to find out whether you can actually run the system as a non-administrator with the help of html properties.

"><script>alert("XSS")</script>&

....

";!="<XSS>=&{()}

ii. Buffer Overflows (BFO)

Why do developers do this all the time? Why can't they think about not putting too many data into it? But they are just looking at it one way, let's put one thousand or two thousand characteristics. Developer doesn't think whether it could be more. We have two developers, one says limit and other is meant to use. As part of the code review, you get a classical buffer overflow, when it comes to the specification and they don't exchange enough data due to lack of communication. So unfortunately, in many software parts, you still have buffer overflows, that are able to compromise such a system can infiltrate with code and fill in something that actually contain data.

"A" *5

.....

"A" *24.000

iii. Integer Overflows (INT)

Integer overflow is something similar. We think normally, we have an entry in certain value range, but we forget this value range can be exceeded by the person putting it or entering it. We use max or min, but it is an integer. Integer is a proxy and we have defined certain data type, for this data type we define limits. When it comes to testing, we stick to these limits.

-1

....

0x10000000

iv. Format String Errors (FSE)

Format string error is something similar; this is about strings, somebody who has understood some programming languages, which are very often used in scriptural science in embedded systems for format indications for text output. Then you know, if you made a mistake their once, nothing will come out at all. Also when it comes to new modern languages, of course format instructions can be used for attacking. Instead of password, such a format string can be used into the system and see how a html browser responds to it.You can try out,if you want to log on to email box, try inputting such formatted strings.

%s%p%x%d

.....

%#01234567x%08x%x%%

v. SQL Injection

There are two types of SQL injection – Passive and Active. We can infiltrate, in one hand, by way of trying to retrieve information that is passive way of SQL injection. The active way of SQL injection is trying to inject changes. Both these thing as bad, we should actually protect data from such attacks.

vi. LDAP Injection

Possibility by way of protocols, we use LDAP as an example only, we can test some information as a user. We never notice somebody can use the LDAP string to test themselves about information. But browsers make it possible.

%28

....

Admin*) ((!userPassword=*)

vii - XPATH Injection

XML instead of HTML, that is another big topic. XML requires the XML parser, which try to evaluate this language, so that we can use it in the program. Then there is more popular and that is XPATH, highly popular, where by way of XPATH parser, we get the text structure by this XPATH parser. Then it is possible that the browser works without memory has this XML tree in this memory for initial period, if we can access this memory, then we can make changes there.

'+or+'1'=1

....

Count (/child::node ())

viii - XML Injection

XML injection is something that we are trying by way of XML language, to do something most browsers understand XML and developers never thought of switching it off.

<?xml version="1.0" encoding = "ISO-8859-1"?><foo>

....

<!ENTITY xxe SYSTEM "title//c:"

c - Fuzzing Advantages

We actually get things, which were not known beforehand, therefore getting zero-day vulnerabilities. Also, since we have not looked at structure of the system, we will not be using just old experience but an intelligent automatic system instead. All of these automatic systems keep learning so we can enter new things for testing. Fuzzing is a fantastic technology but of course we have to keep this fuzzer intelligent.

d - Fuzzing Disadvantages

When everybody does automation, it tries not to just map things that are unknown but everything else as well. This is high effort in maintaining the intelligent fuzzer because this is too late as well as we are in the end of the development cycle. If we find something, then this will become very expensive. This is what keeps some of the customers from using the fuzzy technique extensively.

Conclusion

So what we need is for security testing, address security test activities across entire life cycle of the project. If we want to put safety, then we can do security in parallel. We can do in one go, because both have the same objective of minimizing the number of weak points in the system.

About the Author

Allappagoud has over 15 years of experience in testing, automation, developing and delivering solutions in the domains of consumer electronics and automotive. Allappagoud is always on track with emergence and advancements of testing and test automation methodologies, assessing business prospects, conceiving solutions.

About Sasken

Sasken is a specialist in Product Engineering and Digital Transformation providing concept-to-market, chip-to-cognition R&D services to global leaders in Semiconductor, Automotive, Industrials, Smart Devices & Wearables, Enterprise Grade Devices, SatCom, and Transportation industries. For over 29 years and with multiple patents, Sasken has transformed the businesses of over a 100 Fortune 500 companies, powering over a billion devices through its services and IP.

SASKEN

Systematic Approach and Guidelines for Cyber Security Testing

marketing@sasken.com | www.sasken.com

USA | UK | FINLAND | GERMANY | JAPAN | INDIA

Jan 2019